

RESEARCH ARTICLE

Code Clone Detection Techniques Based on Large Language Models

AFNAN A. ALMATRAFI^{1,2}, FATHY A. EASSA¹, AND SANAA A. SHARAF¹¹Computer Science Department, Faculty of Computing and Information Technology, King Abdulaziz University, Jeddah 21589, Saudi Arabia²Computer Department, Jamoum University College, Umm Al-Qura University, Makkah 21955, Saudi Arabia

Corresponding author: Afnan A. Almatrafi (aaalmatrafi@uqu.edu.sa)

ABSTRACT Code duplication, commonly known as code cloning, is a persistent challenge in software development. While reusing code fragments boosts productivity, excessive cloning poses challenges to maintenance and elevates the risk of bugs. Therefore, integrating code clone detection into the development process is crucial. The extensive code-related knowledge inherent in Large Language Models (LLMs) renders them high-potential candidates for addressing diverse software engineering challenges. However, the effectiveness of LLMs in the specific task of code clone detection requires precise evaluation. This paper proposes an innovative methodology leveraging few-shot instruction-tuned GPT-3.5 Turbo and GPT-4 to detect code clones across all types, focusing on complex clones (Type-3 and Type-4). Unlike conventional approaches confined to specific language pairs or tasks, our method employs versatile language models, showcases generalization strengths for semantic understanding, and leverages instruction tuning with few-shot inference for task-specific adaptability in code clone detection. A conversational dataset was crafted from BigCloneBench for instruction tuning, enhancing task alignment and performance. This study evaluates the proficiency of LLMs in identifying code clones, analyzing the impact of instruction tuning, and assessing the efficiency across various clone types. Experimental results demonstrate these models achieving competitive performance against existing tools for overall and complex clone detection. Integration into an Integrated Development Environment (IDE) enables real-time detection and automated refactoring, bridging the gap between theoretical advancements and practical usability. This work highlights the potential of generalized LLMs setting a new standard in a field traditionally dominated by specialized tools and demonstrates their adaptability for complex challenges in code analysis and maintainability.

INDEX TERMS Clone detection, code duplication, fine-tuning, instruction tuning, large language models, natural language processing, transformers.

I. INTRODUCTION

Software plays an important role in our current lives. Building quality software requires adherence to software engineering principles. One of these principles is code reuse, which emphasizes writing code once and reusing it as needed. However, achieving this principle is often challenged by producing code duplication, or code clones within software projects. Code clones can degrade software quality, leading to increased maintenance costs and higher bug risks, making code clone detection a critical task in software engineering.

The associate editor coordinating the review of this manuscript and approving it for publication was Antonio Piccinno¹.

As a result, code clone detection is one of the active research problems that software engineering researchers are tackling. Traditional approaches to clone detection include text-based [1], [2], token-based [3], [4], and abstract syntax tree approaches [5], [6]. These techniques have proven effective in detecting exact (Type-1) and syntactic (Type-2 and Type-3) clones. However, more complex Type-3 clones remain challenging for some of these approaches, and Type-4 clones, where code fragments differ in syntax but perform the same functionality, are particularly difficult to identify.

Large Language Models (LLMs) have played a transformative role in software engineering, addressing the limitations of traditional approaches. Initially developed to

advance Natural Language Processing (NLP) tasks [7], LLMs have demonstrated the potential to process and understand programming languages, unlocking new opportunities for solving complex software engineering problems. In the realm of code clone detection, this progression has led to the development of specialized models tailored for both Natural Language (NL) and Programming Language (PL) tasks [8].

In this work, we propose the use of GPT-4 [9] and GPT-3.5 Turbo [10], state-of-the-art general-purpose large language models, for code clone detection, leveraging their robust generalization capabilities and adaptability through few-shot instruction tuning. Unlike specialized models designed specifically for code-related tasks, GPT-4 demonstrates broader applicability, revealing pivotal distinctions in its approach to code clone detection. This study not only highlights the promise of GPT-4 in detecting complex clones but also emphasizes the potential of general-purpose LLMs to excel in domains traditionally dominated by specialized models. Furthermore, we integrate the instruction-tuned model into a framework that is seamlessly embedded within an Integrated Development Environment (IDE). This integration enables real-time clone detection and automated refactoring, enhancing the model's practical usability by incorporating it directly into the developer's workflow and providing immediate insights during software development. While the specific contributions of this real-time IDE integration will be detailed in future research, this approach underscores the practical relevance of our model and its potential to streamline modern software engineering practices.

A. MOTIVATION FOR THIS RESEARCH

Our research is motivated by the need for a flexible, scalable solution that can effectively handle both syntactic and semantic clones in real-world software development environments. Existing approaches primarily rely on domain-specific LLMs, such as CodeBERT [8], or fine-tuned models trained extensively on code-related datasets. While these models excel at detecting specific types of clones, they present significant challenges: they require substantial computational resources, rely heavily on large labeled datasets that are difficult and expensive to construct, and often lack adaptability across multiple software engineering tasks.

These limitations motivate the need for a paradigm shift in how code clone detection models are developed and deployed. Instead of creating highly specialized models for every individual software engineering problem, we explore the potential of general-purpose LLMs, such as GPT-4 [9]. GPT-4's robust generalization capabilities and adaptability through few-shot instruction tuning, enabling it to reduce reliance on extensive labeled datasets and minimize the overhead of dataset collection and fine-tuning.

Furthermore, by integrating GPT-4 into a framework embedded within an IDE, we aim to align research progress with practical implementation within software development environments. This integration allows for real-time clone

detection and automated refactoring, addressing both the technical and practical challenges of software development workflows. Ultimately, this work investigates whether adapting general-purpose LLMs can provide scalable, cost-effective, and versatile solutions, positioning them as an alternative to the specialized models dominating current research.

B. WHY USE GENERATOR-BASED MODELS FOR CODE CLONE DETECTION?

In this study, we utilize GPT-4 and GPT-3.5 Turbo generator-based LLMs known for their generalization capabilities and robust performance in natural language tasks. Unlike classifier-based models, which are trained specifically for downstream tasks, generator-based models like GPT are trained using an auto-regressive objective, allowing them to predict the next token in a sequence based on the context. This training paradigm equips GPT with a deep understanding of sequential dependencies and contextual relationships, making it particularly effective for tasks requiring semantic analysis, such as code clone detection.

Additionally, classifier-based models often struggle to detect clones that differ significantly in syntax but share identical functionality, a challenge GPT overcomes through its extensive training on diverse datasets, including programming languages, which enables it to capture complex patterns and recognize functional similarities between code snippets. This ability to generalize across varied contexts is crucial for detecting semantic clones, where code fragments may differ in syntax but perform the same function.

C. CONTRIBUTIONS OF THIS RESEARCH

The main contributions of this study are as follows:

- **General-purpose LLMs vs. Domain-specific LLMs:** Unlike prior studies that rely on domain-specific LLMs like CodeBERT [8], our research demonstrates the viability of using a general-purpose LLM (GPT-4) for code clone detection. By leveraging few-shot instruction tuning, GPT-4 achieves competitive performance without requiring large, specialized datasets, showcasing the versatility of generalized models in software engineering tasks.
- **Few-shot Instruction Tuning:** We introduce an innovative application of few-shot instruction tuning, a lightweight yet effective strategy that allows GPT-4 to adapt to the task with minimal labeled data. This approach offers a scalable and efficient alternative to traditional fine-tuning methods, reducing the overhead of data preparation and model training.
- **Real-time IDE Integration:** While previous studies primarily evaluate models in offline or batch-processing contexts, our framework integrates the instruction-tuned model directly within an IDE. This enables real-time clone detection with automated refactoring, seamlessly connecting research with practical applications.

By providing immediate feedback and actionability, this integration enhances developer workflows and demonstrates the practical applicability of our approach.

The paper is organized as follows: Section II provides a detailed background on transformers and LLMs, including their fine-tuning and instruction fine-tuning techniques. Section III presents a literature review that positions our work within the broader field of code clone detection. Section IV outlines our methodology for leveraging LLMs in code clone detection. Section V presents the results and discussion, highlighting our experimental findings, and Section VI concludes with a discussion on the significance of our findings.

II. BACKGROUND

In this section, we introduce fundamental concepts crucial to this research, covering Transformers, LLMs, the relationship and distinctions between LLMs and Transformers, and the fine-tuning processes, including both general fine-tuning and instruction fine-tuning, and the foundational concepts of code cloning.

A. TRANSFORMERS

Transformers have emerged as a pivotal topic in modern discussions within deep learning. Proposed by a team of researchers at Google in their seminal paper “Attention is all you need” [11], the term “Transformer” signifies a model’s extensive use of the Self-Attention Mechanism. This mechanism enables the model to focus on various parts of the input sequence, departing from the sequential processing of earlier models like Recurrent Neural Networks (RNNs). Transformers employ a sophisticated architecture comprising encoder and decoder layers, multi-head attention, and feedforward networks [12].

Models, such as BERT, GPT, and T5, are transformers that efficiently decipher relationships between words in a sentence or text sequence through parallel processing. They have significantly impacted diverse applications in natural language processing, including language translation, sentiment analysis, and text summarization [13].

1) OVERVIEW OF TRANSFORMER ARCHITECTURE COMPONENTS

The transformer architecture is distinguished by its innovative components, each contributing to its efficiency and effectiveness in processing sequential data, especially in natural language processing tasks. Figure 1 provides a high-level overview of transformer architecture elements [11]. One key element is the Encoder-Decoder Structure, a fundamental design that enables the transformer to handle input and output sequences effectively. The encoder processes the input sequence, while the decoder generates the corresponding output sequence, making it particularly well suited for sequence-to-sequence tasks such as machine translation.

To address the challenge of capturing the sequential order of tokens, the transformer incorporates Positional Encoding. This incorporation is crucial for maintaining the temporal relationships between elements in the sequence, ensuring that the model can understand the input’s sequential nature. The heart of the transformer’s capability lies in the self-attention mechanism, which replaces traditional recurrent and convolutional layers. This mechanism allows each element in the sequence to focus on different parts, capturing long-range dependencies. The introduction of Multi-Head Attention further enhances this capability, as multiple parallel self-attention mechanisms, or “heads” collectively contribute to a richer representation of relationships within the data.

The Feedforward Neural Network component follows the self-attention mechanism, introducing non-linearity to the model. This neural network processes each position independently, capturing complex patterns in the data. Additionally, Layer Normalization and Residual Connections are employed to stabilize training. Residual connections, specifically, facilitate the flow of gradients through the network, aiding in the training of deep models. Training the transformer involves utilizing a Sequence-to-Sequence Objective Function where the model predicts the next element in the sequence. This objective is often complemented by additional objectives, such as attention regularization, to enhance the learning process.

In summary, the architecture components of the transformer - the Encoder-Decoder Structure, Positional Encoding, Self-Attention Mechanism, Multi-Head Attention, Feedforward Neural Network, and the integration of Layer Normalization and Residual Connections - collectively contribute to its ability to capture intricate patterns, dependencies, and long-range relationships within sequential data, making it a foundational model in various applications, particularly in natural language processing.

B. LARGE LANGUAGE MODELS

The LLMs, which are a specialized category of transformers, excel at predicting subsequent words in a sentence based on its context [11]. Notable models include OpenAI’s GPT3.5 Turbo [10] and GPT-4 [9], BigScience’s BLOOM [14], Google’s LaMDA [15] and Meta AI’s LLaMa [16] trained on extensive text corpora, LLMs prove valuable for tasks such as auto-completion, translation, summarization, creative writing, and chatbot interactions [17].

1) THE RELATION AND DIFFERENCES BETWEEN LLMs AND TRANSFORMERS

While transformers and LLMs are interconnected, LLMs represent a subset of transformers specialized in text generation and natural language understanding [18]. Transformers serve a wide array of applications, whereas LLMs, with a primary focus on language-related tasks, are fine-tuned specifically for language modeling and text generation [17]. Ongoing research continues to expand the capabilities of

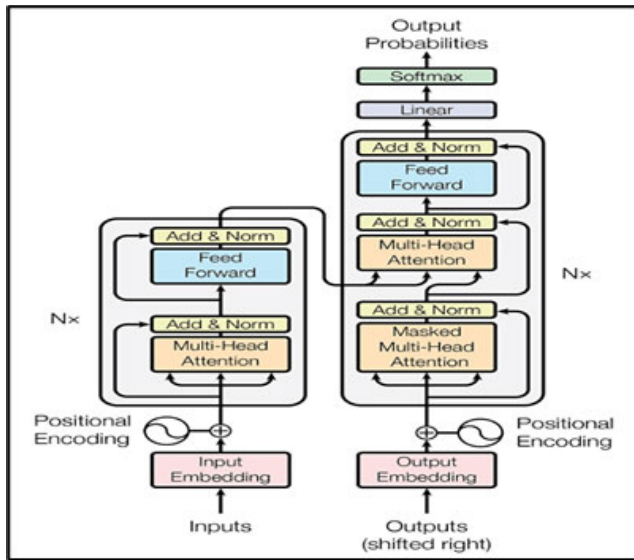


FIGURE 1. High-level overview of transformer architecture [11].

both transformers and LLMs, emphasizing their dynamic nature in addressing evolving challenges in natural language processing [18].

C. LLM FINE-TUNING

Within the domain of LLMs, fine-tuning stands as a pivotal process, serving as the gateway to refining pre-trained models on specialized datasets and optimizing their performance for specific tasks or domains [19]. Before fine-tuning, various techniques were employed directly within the user prompt to enhance the model's output and customize it according to user needs. Two significant methods include in-context learning and zero, one, and few-shot inference [20]. In-context learning enhances the prompt by integrating specific task examples directly within it, providing LLM with a clear blueprint of the intended objectives [21]. Zero-shot inference entails inputting data to the prompt without additional examples [22]. If zero-shot inference is inadequate, methods such as 'one-shot' or 'few-shot inference' can be employed. These methods entail incorporating one or more complete examples within the prompt, thereby assisting smaller LLMs in achieving improved performance.

LLM fine-tuning involves refining pre-trained models on smaller, specific datasets to enhance performance in particular tasks or domains. Techniques such as in-context learning, zero/one/few-shot inference, and general fine-tuning are crucial components of this process. The proposed approach serves to bridge the disparity between general pre-trained models and the unique requirements of specific applications, thereby guaranteeing a closer alignment between the language model and human expectations.

D. LLM INSTRUCTION FINE-TUNING

One approach employed to enhance a model's performance across a range of tasks is through the process of Instruction Fine-Tuning. It involves updating pre-trained language models using labeled data specific to a task. Unlike unsupervised methods, this process employs checked data. During Instruction Fine-Tuning, the model is provided with a dataset of labeled examples formatted as prompt-response pairs [23]. The model's weights are then adjusted based on this supervised learning approach, resulting in a better alignment with specific tasks. This iterative process ensures that the LLM continues to evolve and adapt to diverse tasks, enhancing its versatility and practical applicability.

Finally, to contextualize the application of LLMs in this study, we provide an overview of the concept of code cloning and its various types.

E. CODE CLONING

Code cloning refers to the practice of duplicating code fragments within or across software systems, often resulting from copy-paste programming or deliberate code reuse. While code cloning can accelerate development in the short term, it introduces significant long-term challenges, including increased maintenance costs, higher bug risks, and reduced system efficiency. Consequently, identifying and addressing these clones is critical for ensuring software quality and maintainability.

Code clones are broadly classified into four main types based on their similarities and differences [24]:

- **Type-1 (Exact Clones):** These are identical code fragments except for variations in whitespace or comments. They are the simplest form of clones and are typically the easiest to detect.
- **Type-2 (Renamed Clones):** These clones retain identical code structures but differ in naming conventions, such as variable or function names. Detecting Type-2 clones requires models that can understand context beyond simple token matching.
- **Type-3 (Near-Miss Clones):** These code snippets exhibit structural similarity but may include minor modifications, such as added, removed, or altered statements. Detecting Type-3 clones requires a deeper understanding of code structure and semantics.
- **Type-4 (Semantic Clones):** These clones represent the most challenging type, as they perform the same functionality but are implemented with different syntax and structure. Detecting these clones requires an advanced understanding of the functional intent of the code.

To evaluate the performance of clone detection models, researchers commonly use benchmarks like BigCloneBench [25], a dataset that provides labeled examples of all clone types. This dataset serves as a comprehensive resource for testing models across varying levels of similarity, from exact matches to complex semantic equivalence.

III. LITERATURE REVIEW

Identifying and mitigating code clones is crucial for enhancing software reliability and facilitating efficient development processes. While traditional methods of code clone detection often rely on handcrafted features and specific heuristics, the evolving landscape of software development necessitates more sophisticated approaches. Recent advancements in deep learning have shown promise in addressing the nuances of code clone detection. As our proposed approach resides within the domain of deep learning techniques, this section outlines the evolution and diversity of clone detection methods over the years, with particular emphasis on deep learning approaches.

Li et al. introduced CCLEARNER in 2017, a token-based clone detection approach that utilizes deep learning [26]. This approach deviates from previous methods, which were specific to certain types of clones. CCLEARNER employs deep learning on the method-level, using both code clones and non-clones as positive and negative examples to train its model. It trains a binary class classifier by deducing commonalities and potential variations between the peers of known clone method pairs. Subsequently, it extracts similarity vectors for both types of examples and feeds them to a deep neural network (DNN). CCLEARNER then utilizes this binary classifier to compare methods pair-by-pair within a given codebase, effectively identifying clones. The findings of the paper demonstrated promising results, as CCLEARNER achieved competitive clone detection effectiveness at a low time cost compared to approaches that did not leverage deep learning.

Zhao et al. presented CDLH (Clone Detection with Learning to Hash) in 2017, a framework designed to detect functional clones in software [27]. It is a comprehensive deep feature learning framework that uses AST-based Long Short-Term Memory (LSTM) to derive representations of code fragments and hash function parameters. It enhances the comparison of code fragments as it takes into account both the lexical and syntactical data of source codes. By addressing the limitations of traditional methods that rely on handcrafted features based on lexical or syntactical information, CDLH utilizes supervised deep learning, where the learning process is guided by labelled examples of code. This approach formulates clone detection as a supervised learning to hash problem, allowing code to be transformed into hash codes for rapid functional similarity assessment. Experiments on software clone detection benchmarks demonstrated that CDLH surpasses current state-of-the-art approaches in identifying functional clone pairs in addition to its applicability to the other types of clones.

Zhang et al. proposed a new method in 2019 for Type-3 code clone detection [28]. They start by using tree embedding to generate a vector for each intermediate node in the Abstract Syntax Tree (AST), which captures the structural information of the AST. These node vectors are then combined to form a tree vector using a lightweight method. The key step involves

comparing the tree vectors using Euclidean distances to detect code clones. They developed a tool called TECCD based on this approach and tested it using the BigCloneBench [25] as well as seven other large-scale Java projects. The researchers opted for a lightweight embedding method to derive the AST vector from node vectors, which proved to be more efficient than RNN-based methods. Importantly, unlike existing techniques that rely on project-specific language to train deep learning models, their method does not depend on lexical information during the learning process, allowing the trained node vector representation to be broadly applicable across different projects. The results showed that this approach achieves high accuracy and recall, outperforming existing methods.

In the same year, Chen et al. introduced Tree-Based Convolution for Clone Detection (TBCCD) for detecting semantic code clones using deep learning [29]. This method employs tree-based convolution on token-enhanced ASTs in order to capture both structural and lexical information from code fragments. Unlike previous methods, this approach does not require transforming an AST into a complete binary tree and demonstrates superior effectiveness and efficiency compared to the state-of-the-art CDLH approach. A key feature of this method is the introduction of a new embedding technique called Position-Aware Character Embedding (PACE). PACE represents each token as a position-weighted combination of characters one-hot embeddings. The TBCCD uses a tree-based Convolutional Neural Network (CNN) classifier, which processes pairs of code fragments simultaneously with shared parameters, focusing on minimizing the cosine similarity for non-clone pairs and maximizing it for clone pairs. This deep learning approach has demonstrated an improvement in detecting semantic code clones, outperforming CDLH with increases in F1 scores on BigCloneBench benchmarks [25].

Feng et al. in 2020, marked a significant advancement in transformer-based models by introducing CodeBERT, derived from the BERT and RoBERTa architectures [8]. By leveraging transformer-based neural networks, CodeBERT adeptly captures contextual embeddings, allowing for a nuanced understanding of similarities and variations within code fragments. Its bimodal pre-training in programming language and natural language has enabled it to excel in various downstream applications. In the realm of clone detection, CodeBERT shows promise by enhancing the accuracy of identifying code clones through its unique pre-training approach. It harnesses vast amounts of code data to recognize complex patterns that may escape traditional methods.

Liu et al. introduced SSCD in 2022, a BERT-based clone detection method targeting Type 3 and Type 4 clones in large codebases [30]. SSCD computes embeddings for each code fragment using a BERT-based neural network, fine-tuned with contrastive loss, and identifies similar fragments through a k-nearest neighbor (k-NN) search. This method avoids the

inefficiency of pairwise comparisons in neural network-based clone detection and demonstrates better efficiency and effectiveness compared to traditional methods like SAGA [31] and SourcererCC [32]. SSCD tackles scalability for GPU; as it can analyze millions of lines of code quickly. The approach shows significant recall improvement and faster execution times, marking a substantial advancement in scalable clone detection.

Recent years have also witnessed the evaluation of ChatGPT and other LLMs for code clone detection. Wang et al. in 2023 conducted an empirical evaluation of ChatGPT's capability in code clone detection, focusing on its effectiveness in identifying various types of code clones [33]. The study highlighted ChatGPT's ability to understand and explain code semantics in simpler cases but revealed its limitations in handling complex scenarios, particularly with semantically similar code fragments. The research emphasizes the challenges ChatGPT faces in accurately detecting semantic clones, suggesting a need for advancements to address these shortcomings.

In 2024, Khajezade, Fard, and Shehata assessed the generalizability of code clone detection models in few-shot learning scenarios [34]. Their study evaluated the performance of CodeBERT [8] and TBCCD [29] on unseen programming problems, unseen languages, and combinations of both. They demonstrated that using Model-Agnostic Meta-Learning (MAML) significantly improved the models' performance in few-shot settings across Java, C++, and Ruby. However, the models struggled to maintain high accuracy for underrepresented languages like Ruby and more complex scenarios, highlighting the need for further improvements in few-shot learning methods for code clone detection.

Khajezade et al. also in 2024 investigated the efficacy of LLMs, specifically ChatGPT, in detecting code clones [35]. The study employed ChatGPT in a zero-shot setting to detect Type-4 code clones across Java and Ruby code pairs. The results showed that ChatGPT achieved an F1-score of 0.877 in cross-language clone detection, outperforming baseline models like CodeBERT [8] and GraphCodeBERT [36]. However, the study also highlighted challenges in prompt design and problem complexity, which significantly influenced the model's performance.

Zhang and Saber evaluated the performance of GPT-3.5 and GPT-4 in code clone detection in 2024 [37]. Using datasets comprising human-generated and LLM-generated code pairs, they demonstrated that GPT-4 consistently outperformed GPT-3.5 across various clone types. Nevertheless, both models exhibited limitations in detecting complex Type-4 clones, underscoring the need for further advancements in LLM-based methodologies for code clone detection.

Khajezade et al. [35] and Zhang et al. [37] demonstrated the use of domain-specific LLMs, such as CodeBERT, and general-purpose LLMs, such as GPT-4, for code clone detection. Khajezade et al. [34] focused on fine-tuning domain-specific models in few-shot learning scenarios,

showcasing strong performance but highlighting limitations such as reliance on large labeled datasets and computationally intensive fine-tuning. Zhang et al. [37], on the other hand, evaluated GPT-4 in detecting various clone types, demonstrating its superiority over earlier models like GPT-3.5 but also noting challenges in detecting complex semantic clones. Wang et al. [33] further explored ChatGPT's capabilities for detecting semantic clones in offline scenarios, identifying its potential but also its inconsistency without task-specific alignment or real-time applicability.

In contrast, our study leverages a general-purpose LLM (GPT-3.5 Turbo and GPT-4), adapted with few-shot instruction tuning, representing a paradigm shift that minimizes reliance on extensive labeled datasets while maintaining competitive performance across all clone types (Type-1 to Type-4). By guiding the model with task-specific prompts, our approach streamlines the adaptation process and enhances task alignment with minimal examples, addressing scalability and resource efficiency challenges faced by domain-specific and fine-tuned methods. Furthermore, unlike prior studies that focus on offline evaluations, our framework integrates the instruction-tuned model into an IDE environment, enabling real-time code analysis and automated refactoring. This integration connects theoretical progress with the practical needs of developers, an aspect largely unexplored in existing works.

Compared to traditional approaches such as CCLEARNER [24], CDLH [25], TECCD [26], TBCCD [28], and SSCD [29], which focus on specific methodologies such as token-based or tree-based approaches, our approach employs a generalized model that unifies lessons learned from these methods while going beyond their constraints. Unlike task-specific models like CodeBERT [8], which excel in targeted scenarios but lack broader adaptability, our work demonstrates the versatility of general-purpose LLMs in addressing diverse clone detection challenges. By covering all clone types and achieving real-time usability, our approach sets a new precedent in code clone detection, emphasizing efficiency, scalability, and practical integration into modern software development workflows, as summarized in Table 1.

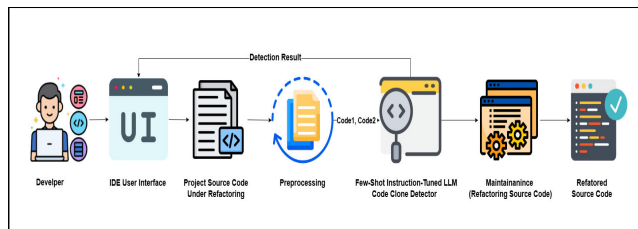
IV. METHODOLOGY

In this study, we propose a framework specifically designed for developers working within the IDE workflow. As illustrated in Figure 2, the process begins with developers using the IDE, where the project source code undergoes a preprocessing step at the method level, resulting in the creation of snippets. Each snippet represents a distinct portion of the code, and two such snippets are then utilized as input for our few-shot instruction-tuned code clone detector. This preprocessing step is crucial to ensure the effective representation of the source code for accurate and efficient code clone detection.

The detector, described in detail below, leverages few-shot instruction tuning to identify potential code clones. The identified clones advance to a maintenance step, an enhancement

TABLE 1. Summary of literature review.

Year	Name	Classifier	Preprocessing	Targeted Clone Types
2017	Ccleanner [26]	DNN	Tokens	Type 1,2,3,4
2017	CDLH [27]	LSTM	AST	Type 4
2019	TECCD [28]	RNN	AST	Type 3
2019	TBCCD [29]	CNN	AST and Token	Type 4
2020	CodeBERT [8]	CodeBert	Tokens	Type 3,4
2022	SSCD [30]	CodeBert	Tokens	Type 3,4
2023	ChatGPT (Zero-Shot) [33]	ChatGPT	Prompts	Type 4
2024	MAML (Few-Shot) [34]	MAML	Few-Shot Dataset	Type 1,2,3
2024	ChatGPT (Zero-Shot) [35]	ChatGPT	Prompts	Type 4
2024	GPT-4 [37]	GPT-4	Prompts	Type 1,2,3,4
2024	Proposed approach (Few-Shot) [This study]	Instruction-tuned LLM GPT-3.5 Turbo and GPT-4	Few-Shot Conversational Dataset	Type 1,2,3,4

**FIGURE 2.** Overview of the proposed framework.

phase where the code undergoes refactoring to eliminate the clones, thereby improving code quality. The result of this process is a refactored codebase, representing a more optimized version rectified from the clones identified during the detection stage. The framework seamlessly integrates into the developer's workflow, enhancing code quality and maintainability.

A. INSTRUCTION TUNING AND GENERATOR-BASED MODELS

The core of our methodology is the application of instruction tuning, a fine-tuning strategy that adapts a pre-trained general-purpose LLM to a specific task using labeled examples formatted as prompt-response pairs. Unlike traditional fine-tuning, which often requires extensive labeled datasets and primarily focuses on syntactic analysis, instruction tuning enables the model to align with task-specific requirements through minimal labeled examples. This strategy is particularly effective for code clone detection, as it trains the model to recognize functional equivalence between code snippets, even when their syntax varies significantly.

The choice of a generator-based model like GPT-4 and GPT-3.5 Turbo is driven by its auto-regressive training paradigm, which equips it with the ability to model long-range dependencies and contextual relationships. Unlike classifier-based models, which focus on predefined categories, GPT excels at identifying semantic similarities between code snippets. By leveraging its pre-trained knowledge of both natural and programming languages,

GPT-4 adapts seamlessly to specialized tasks like code clone detection, particularly for challenging cases such as Type-4 clones (semantic clones).

1) TARGETED CLONE TYPES

Our methodology is designed to target all four types of code clones:

- Type-1: Exact clones, where code fragments are identical.
- Type-2: Renamed clones, where only variable names or comments are altered.
- Type-3: Near-miss clones, which involve more complex structural changes but retain functional equivalence.
- Type-4: Semantic clones, where code fragments differ syntactically but achieve the same functionality.

Few-shot instruction tuning enables the model to capture both syntactic patterns (Type-1 and Type-2 clones) and deeper complex and semantic similarities (Type-3 and Type-4 clones). Task-specific prompts include labeled examples of these clone types, allowing the model to distinguish between minor structural changes and meaningful functional equivalence. By targeting the full spectrum of clone types, the methodology ensures comprehensive coverage while addressing real-world software development challenges.

B. PHASES OF THE METHODOLOGY

Our approach is structured into two interconnected phases:

1) PROMPT ENGINEERING PHASE

In this initial phase, the goal is to transform the well-known BigCloneBench dataset [25] into a format that is suitable for interaction with the general-purpose models used in this study, GPT-3.5 Turbo and GPT-4. Task-specific instructions from BigCloneBench were crafted into a conversational dataset of prompt-response pairs. This dataset adapts raw code snippets and their clone status into a dialogue-based format that mimics user interactions, such as querying GPT about the clone status of two code snippets. The outcome of this task is a new dataset where each entry is a structured prompt that simulates a conversational scenario for code clone detection. This approach aligns the model with real-world software engineering workflows by simulating natural queries about clone detection.

2) INSTRUCTION TUNING PHASE

Building on the tailored dataset, we applied instruction fine-tuning to GPT-3.5 Turbo and GPT-4 with a focus on code clone detection. The instruction tuning was conducted using OpenAI's fine-tuning platform.

Instruction tuning is a fine-tuning strategy that adapts a pre-trained general-purpose LLM to a specific task by aligning the model's behavior with task-specific requirements. Unlike traditional fine-tuning, which requires extensive labeled datasets and primarily focuses on syntactic patterns, instruction tuning leverages minimal labeled examples to

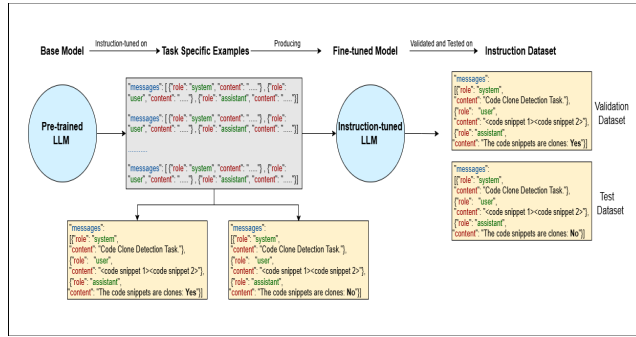


FIGURE 3. Instruction tuning GPT-4 for code clone detection.

enable the model to detect functional similarities between code snippets, even when their syntax varies significantly.

The tuning process involved an iterative refinement of the prompts, incorporating diverse code examples to improve the model's ability to analyze pairs of code snippets accurately and determine their clone status. This process showcased the model's few-shot learning capabilities, allowing it to generalize from a limited number of labeled examples. Figure 3 illustrates the process of instruction tuning GPT-4 for the code clone detection task.

This phase focused on enhancing the model's capacity to detect both syntactic clones (Type-1 and Type-2) and semantic clones (Type-3 and Type-4), with particular emphasis on challenging semantic clones. By leveraging the cloud infrastructure provided by OpenAI, the fine-tuning process ensured high precision and recall across diverse clone types.

3) ADVANTAGES OF INSTRUCTION TUNING:

- **Task Alignment:** Explicit task-specific instructions aligned the model with the requirements of code clone detection.
- **Flexibility and Generalization:** Few-shot learning reduced reliance on extensive datasets while maintaining robust performance.
- **Enhanced Contextual Understanding:** Enabled the detection of semantic clones that differ syntactically but share functional equivalence.

After instruction tuning, the model was evaluated using Google Colab, a cloud-based platform with GPU acceleration (NVIDIA T4 GPU). The evaluation involved running inference on a subset of the BigCloneBench dataset to measure performance across different clone types. Metrics such as precision, recall, and F1 score were calculated for the overall dataset as well as specific clone types (Type-3 and Type-4), demonstrating the model's ability to generalize to unseen examples while maintaining high performance.

This phase not only validated the model's effectiveness in detecting clones but also highlighted its practical usability for real-world software engineering tasks. By testing on challenging semantic clones, the evaluation phase ensured that the model's predictions aligned with the functional equivalence required in software development workflows.

4) DATASET AND ENVIRONMENT

We trained and evaluated our model using the BigCloneBench dataset [25], a widely recognized benchmark containing labeled examples for all four clone types. The dataset's comprehensive coverage ensured robust training and evaluation for both syntactic and semantic similarities. The evaluation environment included Google Colab for GPU-accelerated inference and IntelliJ IDEA for real-time integration testing.

5) IDE INTEGRATION

The fine-tuned model was integrated into IntelliJ IDEA as a plugin, enabling real-time code clone detection and automated refactoring. The plugin was tested on a development machine with an Intel i7 processor, 16 GB RAM, and Java 11. This integration allows developers to seamlessly identify and resolve code clones within their workflow, connecting research-driven innovations with real-world applicability.

V. RESULTS AND DISCUSSION

In this phase of our research, we conduct experiments to assess the performance of our proposed code clone detector within the IDE workflow. The primary objective is to evaluate the LLM's effectiveness in accurately identifying code clones, which is represented in our framework as a few-shot instruction-tuned code clone detector.

The experiments are specifically designed to assess the detector's capabilities in code clone identification and the impact of instruction tuning and prompt engineering on the quality of clone detection within the IDE workflow. We aim to understand how the detector performs in real-world coding scenarios and its ability to seamlessly integrate into developers' workflows.

A. EXPERIMENTAL SETUP

The experimental setup involved the following environments:

- **Instruction Tuning Environment:** OpenAI's fine-tuning platform was used for instruction tuning, leveraging cloud-based infrastructure with pre-trained models to adapt the model for the code clone detection task.
- **Evaluation Environment:** Google Colab was employed for evaluation, utilizing an NVIDIA T4 GPU for accelerated inference and performance testing, ensuring efficient and scalable evaluations.
- **Development Environment:** The IntelliJ IDEA integration was tested on a standard development setup with an Intel i7 processor, 16 GB RAM, and Java 11. This configuration ensured that the plugin operated efficiently within typical development workflows.

B. RESULTS BEFORE INSTRUCTION TUNING

To begin, we integrated the GPT-4-based model for the code clone detection task into the framework without instruction tuning. The initial model demonstrated the ability to detect code clones of different types, yielding promising results with

a Precision of 0.67, Recall of 1.00, and F1 Score of 0.80. However, certain limitations emerged when utilizing the GPT model in its raw form:

- 1) **Token Limitations:** Token limits restricted the ability to send the overall source code, requiring the code to be sent in chunks. While this limitation could be mitigated by increasing the token limit, this approach increased costs.
- 2) **Prompt Formatting:** The construction of prompts had a significant impact on the model's accuracy and focus. Different prompt structures generated different responses with varying levels of success in guiding the model's behavior. Explicit instructions produced more accurate responses, while ambiguous or overly complex prompts led to generalized or inconsistent outputs. The prompt's design directly influenced the model's ability to infer clone relationships, emphasizing the need for careful prompt engineering.
- 3) **Response Variability:** The response formats varied, sometimes directly identifying clones while at other times included dense outputs with detection results, explanations, deductions, or suggestions for code improvement, which impacted consistency.

These limitations guided us toward instruction tuning to enhance accuracy, reduce response variability, and tailor the model's output to our specific requirements.

C. RESULTS AFTER INSTRUCTION TUNING

As discussed in Section I and Section IV, we moved towards few-shot instruction tuning for general-purpose LLMs rather than extensive fine-tuning to leverage the learning capabilities and semantic understanding of NL large language models while offering a more flexible and resource-efficient code-related tasks models. The instruction tuning was conducted using a training set of 100 samples and validated with an additional 100 samples, reflecting the concept of few-shot inference. Testing was performed on a larger dataset of 2,000 samples. During the instruction fine-tuning process for both models, training and validation loss steadily decreased, reflecting effective learning from the provided data and contributing to the observed performance improvements.

Using the few-shot instruction-tuned model for code clone detection resulted in significant improvements. The optimized GPT-3.5 Turbo model achieved a Precision of **0.79**, Recall of **0.91**, and F1 Score of **0.84** across all clone types. Additionally, optimized GPT-4 demonstrated improved results with a Precision of **0.83**, Recall of **0.91**, and F1 Score of **0.87** across all clone types, showcasing the scalability and adaptability of the proposed framework.

These enhancements signify the model's improved accuracy in identifying true positives while maintaining a balanced trade-off between precision and recall. Furthermore, task-specific prompts during instruction tuning enhanced the model's ability to learn under different circumstances and focus on detecting both syntactic and semantic clones.

TABLE 2. Focused evaluation of type-3 and type-4 clones using instruction-tuned GPT models.

Clone Type	Instruction-tuned Model	Precision	Recall	F1 Score
Type-3	GPT-3.5 Turbo	0.90	0.88	0.89
Type-3	GPT-4	0.93	0.87	0.90
Type-4	GPT-3.5 Turbo	0.90	0.92	0.91
Type-4	GPT-4	0.93	0.92	0.93

TABLE 3. Comparative analysis of overall clone detection.

Tool	Detected Clone Types	Precision	Recall	F1 Score
NiCad [2]	Type 1, 2	0.95	0.82	0.88
SourcererCC [32]	Type 1, 2, 3	0.93	0.87	0.90
Oreo [38]	Type 1, 2, 3	0.91	0.89	0.90
CodeBERT [8]	Type 3, 4	0.88	0.85	0.86
GraphCodeBERT [36]	Type 3, 4	0.89	0.87	0.88
CodeT5+ [39]	Type 3, 4	0.90	0.88	0.89
Our Instruction-tuned Model (GPT-3.5 Turbo)	Type 1, 2, 3, 4	0.79	0.91	0.84
Our Instruction-tuned Model (GPT-4)	Type 1, 2, 3, 4	0.84	0.91	0.87

TABLE 4. Comparative performance of type-3 and type-4 clone detection with LLM-based tools.

Tool	Precision	Recall	F1 Score
CodeBERT [8]	0.88	0.85	0.86
GraphCodeBERT [36]	0.89	0.87	0.88
CodeT5+ [39]	0.90	0.88	0.89
Our Instruction-tuned Model (GPT-3.5 Turbo)	0.89	0.90	0.89
Our Instruction-tuned Model (GPT-4)	0.94	0.88	0.91

1) FOCUSED EVALUATION ON TYPE-3 AND TYPE-4 CLONES
To provide a comprehensive evaluation, we conducted a focused analysis on Type-3 (near-miss clones) and Type-4 (semantic clones), as these represent the most complex cases of functional equivalence. The results for these clone types are summarized in Table 2.

These metrics demonstrate the model's capability to detect complex clones, where traditional syntactic analysis methods typically fall short.

D. COMPARATIVE ANALYSIS

We compared the performance of our model against existing tools across all clone types. The results highlight the competitive performance of our instruction-tuned models, as illustrated in Table 3.

1) TYPE-3 AND TYPE-4 CLONES COMBINED

The comparative performance for Type-3 and Type-4 clones underscores the effectiveness of our instruction-tuned GPT models, as detailed in Table 4.

E. DISCUSSION

The results indicate that both GPT-3.5 Turbo and GPT-4 models perform competitively across all clone types. Notably, GPT-4 achieved a 5% improvement in precision compared to GPT-3.5 Turbo (Precision: 0.84 vs. 0.79) while maintaining consistent recall (0.91). This improvement highlights GPT-4's ability to better detect functional similarities across a broad spectrum of clone types, particularly Type-3 and Type-4 clones, which are often challenging for traditional tools. While GPT-3.5 Turbo exhibited strong recall and balanced performance, GPT-4 demonstrated higher precision and F1 scores, making it more effective for analyzing complex semantic clones and intricate code relationships.

Overall, both models performed well across all clone types, demonstrating the adaptability of our instruction-tuned approach. A focused evaluation on Type-3 and Type-4 clones revealed the nuanced capabilities of GPT-4. Specifically, GPT-4 achieved a Precision of 0.93 and F1 Score of 0.93 for Type-4 clones, underscoring its ability to capture functional equivalence in code fragments that differ significantly in syntax. This focused performance highlights the effectiveness of instruction tuning in addressing deeper functional relationships, which traditional syntactic analysis methods often miss.

A comparative analysis with existing tools further validates the strength of our approach. For overall clone detection (Type-1 to Type-4), GPT-4 achieved competitive metrics (Precision: 0.84, Recall: 0.91, F1 Score: 0.87) compared to tools like NiCad [2] and SourcererCC [32], which primarily focus on syntactic clones. For Type-3 and Type-4 clones, GPT-4 outperformed existing LLM-based tools such as CodeBERT [8] and CodeT5+ [39] in precision and F1 score, reinforcing its capability to handle semantic clones effectively. GPT-3.5 Turbo also demonstrated competitive performance, particularly in the recall, further showcasing the adaptability of instruction-tuned models.

These findings demonstrate that instruction tuning effectively enhances task alignment by enabling models to detect complex semantic clones with minimal reliance on extensive labeled datasets. By guiding the model with task-specific prompts, our approach offers a scalable method to adapt general-purpose LLMs for specialized software engineering tasks.

The integration of these models into IDE workflows further highlights their practical relevance. Real-time detection and automated refactoring within the IDE enable developers to identify and resolve code clones seamlessly. This practical contribution links advancements in clone detection research with the practical demands of software development workflows, positioning our approach as a scalable and actionable solution for modern software engineering challenges.

VI. CONCLUSION

This paper introduces a novel methodology leveraging instruction-tuned large language models for effective code

clone detection. Our approach applies few-shot instruction tuning to GPT-3.5 Turbo and GPT-4, enabling these general-purpose models to achieve competitive performance across all clone types, including the challenging complex (Type 3 and Type-4) clones. By tailoring task-specific prompts from the BigCloneBench dataset, we demonstrated the ability of instruction tuning to enhance functional understanding and adapt LLMs for code clone detection with minimal labeled examples. Experimental evaluations highlighted the robustness of our methodology, with the instruction-tuned GPT-4 surpassing both traditional tools and existing LLM-based approaches in detecting Type-3 and Type-4 clones, achieving a notable balance of precision and recall. Furthermore, integrating these models into an IDE framework for real-time detection and automated refactoring underscores their practical applicability and scalability. This integration bridges theoretical advancements in LLMs with real-world software engineering workflows, enhancing developer productivity and software maintainability. Our findings emphasize the transformative potential of instruction-tuned LLMs in software engineering. This work lays the foundation for future research in leveraging LLMs to address complex programming tasks and extending this framework to other software engineering domains.

ACKNOWLEDGMENT

The authors would like to express sincere gratitude to King Abdulaziz University for providing the necessary academic and research support and to Umm Al-Qura University for facilitating this scholarship. Chat-GPT [40] has been used in the background and the experiment sections for the purpose of paraphrasing.

REFERENCES

- [1] S. Ducasse, M. Rieger, and S. Demeyer, "A language independent approach for detecting duplicated code," in *Proc. IEEE Int. Conf. Softw. Maintenance (ICSM)*, Aug. 1999, pp. 109–118.
- [2] C. K. Roy and J. R. Cordy, "NICAD: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization," in *Proc. 16th IEEE Int. Conf. Program Comprehension*, Jun. 2008, pp. 172–181.
- [3] T. Kamiya, S. Kusumoto, and K. Inoue, "CCFinder: A multilingualistic token-based code clone detection system for large scale source code," *IEEE Trans. Softw. Eng.*, vol. 28, no. 7, pp. 654–670, Jul. 2002.
- [4] H. A. Basit and S. Jarzabek, "Efficient token-based clone detection with flexible tokenization," in *Proc. IEEE Int. Conf. Softw. Eng.*, Sep. 2007, pp. 513–516.
- [5] L. Jiang, G. Misherghi, Z. Su, and S. Glondou, "Deckard: Scalable and accurate tree-based detection of code clones," in *Proc. IEEE/ACM Int. Conf. Softw. Eng.*, May 2007, pp. 96–105.
- [6] I. D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier, "Clone detection using abstract syntax trees," in *Proc. Int. Conf. Softw. Maintenance*, Nov. 1998, pp. 368–377.
- [7] D. Rothman and A. Gulli, *Transformers for Natural Language Processing: Build, Train, and Fine-Tune Deep Neural Network Architectures for NLP with Python, PyTorch, TensorFlow, BERT, and GPT-3*. Birmingham, U.K.: Packt Publishing, 2022.
- [8] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.
- [9] OpenAI et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [10] GPT-4 Tech. Rep. *GPT-3.5 Turbo*. Accessed: Nov. 3, 2024. [Online]. Available: <https://platform.openai.com/docs/models/gpt-3-5>

- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Jun. 2017, pp. 5998–6008.
- [12] K. Han, A. Xiao, E. Wu, J. Guo, C. Xu, and Y. Wang, "Transformer in transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 15908–15919.
- [13] T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," *AI Open*, vol. 1, no. 1, p. 110, 2022.
- [14] B. Workshop et al., "Bloom: A 176B-parameter open-access multilingual language model," 2022, *arXiv:2211.05100*.
- [15] R. Thoppilan et al., "LaMDA: Language models for dialog applications," 2022, *arXiv:2201.08239*.
- [16] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
- [17] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Barnes, and A. Mian, "A comprehensive overview of large language models," 2023, *arXiv:2307.06435*.
- [18] M. Shanahan, "Talking about large language models," *Commun. ACM*, vol. 67, no. 2, pp. 68–79, Feb. 2024.
- [19] M. Ekman, *Learning Deep Learning: Theory and Practice of Neural Networks, Computer Vision, Natural Language Processing, and Transformers Using TensorFlow*. Boston, MA, USA: Addison-Wesley, 2021.
- [20] J. Shieh. (2023). *Best Practices for Prompt Engineering with OpenAI API*. OpenAI. [Online]. Available: <https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-openai-api>
- [21] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li, H. Xia, J. Xu, Z. Wu, T. Liu, B. Chang, X. Sun, L. Li, and Z. Sui, "A survey on in-context learning," 2022, *arXiv:2301.00234*.
- [22] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models," in *Data Intelligence and Cognitive Informatics*. Cham, Switzerland: Springer, 2023, pp. 387–402.
- [23] S. Zhang, L. Dong, X. Li, S. Zhang, X. Sun, S. Wang, J. Li, R. Hu, T. Zhang, F. Wu, and G. Wang, "Instruction tuning for large language models: A survey," 2023, *arXiv:2308.10792*.
- [24] C. K. Roy, J. R. Cordy, and R. Koschke, "Comparison and evaluation of clone detection techniques and tools: A qualitative approach," *Sci. Comput. Program.*, vol. 74, no. 7, pp. 470–495, May 2009.
- [25] J. Svajlenko, J. F. Islam, I. Keivanloo, C. K. Roy, and M. M. Mia, "Towards a big data curated benchmark of inter-project code clones," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol.*, Sep. 2014, pp. 476–480.
- [26] L. Li, H. Feng, W. Zhuang, N. Meng, and B. Ryder, "CClearner: A deep learning-based clone detection approach," in *Proc. IEEE Int. Conf. Softw. Anal. Evol. Reeng.*, Sep. 2017, pp. 249–260.
- [27] H. Wei and M. Li, "Supervised deep features for software functional clone detection by exploiting lexical and syntactical information in source code," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol.*, Aug. 2017, pp. 3034–3040.
- [28] Y. Gao, Z. Wang, S. Liu, L. Yang, W. Sang, and Y. Cai, "TECCD: A tree embedding approach for code clone detection," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Sep. 2019, pp. 145–156.
- [29] H. Yu, W. Lam, L. Chen, G. Li, T. Xie, and Q. Wang, "Neural detection of semantic code clones via tree-based convolution," in *Proc. IEEE Conf. Neural Netw.*, May 2019, pp. 70–80.
- [30] M. Chochlov, G. A. Ahmed, J. V. Patten, G. Lu, W. Hou, D. Gregg, and J. Buckley, "Using a nearest-neighbour, BERT-based approach for scalable clone detection," in *Proc. IEEE Conf. Scalable Clone Detection*, Oct. 2022, pp. 582–591.
- [31] G. Li, Y. Wu, C. K. Roy, J. Sun, X. Peng, N. Zhan, B. Hu, and J. Ma, "SAGA: Efficient and large-scale detection of near-miss clones with GPU acceleration," in *Proc. IEEE Conf. Softw. Maintenance Evol.*, Feb. 2020, pp. 272–283.
- [32] H. Sajjani, V. Saini, J. Svajlenko, C. K. Roy, and C. V. Lopes, "SourcererCC: Scaling code clone detection to big-code," in *Proc. IEEE/ACM 38th Int. Conf. Softw. Eng. (ICSE)*, May 2016, pp. 1157–1168.
- [33] P. Wang, L. Zhu, Q. Wang, O. Jaiteh, and C. Guo, "An empirical understanding of code clone detection by ChatGPT," in *Proc. 6th Int. Conf. Data Sci. Inf. Technol. (DSIT)*, Jul. 2023, pp. 78–83.
- [34] M. Khajezade, F. H. Fard, and M. S. Shehata, "Evaluating few-shot and contrastive learning methods for code clone detection," *Empirical Softw. Eng.*, vol. 29, no. 6, p. 163, Nov. 2024.
- [35] M. Khajezade, J. J. Wu, F. H. Fard, G. Rodriguez-Perez, and M. S. Shehata, "Investigating the efficacy of large language models for code clone detection," in *Proc. 32nd IEEE/ACM Int. Conf. Program Comprehension*, Apr. 2024, pp. 161–165.
- [36] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. Kun Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, "GraphCodeBERT: Pre-training code representations with data flow," 2020, *arXiv:2009.08366*.
- [37] Z. Zhang and T. Saber, "Assessing the code clone detection capability of large language models," in *Proc. 4th Int. Conf. Code Quality (ICQ)*, Jun. 2024, pp. 75–83.
- [38] V. Saini, F. Farmahinifarahani, Y. Lu, P. Baldi, and C. V. Lopes, "Oreo: Detection of clones in the twilight zone," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Oct. 2018, pp. 354–365.
- [39] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.
- [40] OpenAI. *ChatGPT*. [Online]. Available: <https://www.openai.com/chatgpt>

AFNAN A. ALMATRAFI received the B.S. and M.S. degrees in computer science from Umm Al-Qura University, Saudi Arabia, in 2015 and 2019, respectively. She is currently pursuing the Ph.D. degree with the Computer Science Department, Faculty of Computing and Information Technology, King Abdulaziz University, Saudi Arabia. She is also a Lecturer with Umm Al-Qura University. Her current research interests include software engineering, deep learning, large language models, and agent-based software engineering.



FATHY A. EASSA received the B.Sc. degree in electronics and electrical communication engineering from Cairo University, Egypt, in 1978, the M.Sc. degree in computers and systems engineering from Al-Azhar University, Cairo, Egypt, in 1984, and the joint Ph.D. degree in computers and systems engineering from Al-Azhar University, joint supervision with the University of Colorado, USA, in 1989. He is currently a Full Professor with the Computer Science Department, Faculty of Computing and Information Technology, King Abdulaziz University, Saudi Arabia. His research interests include agent-based software engineering, the IoT security, software engineering, big data management and security, distributed systems security, and exascale systems testing.

SANAA A. SHARAF received the B.Sc. degree in computer science from King Abdulaziz University, Jeddah, Saudi Arabia, in 1997, the M.Sc. degree (Hons.) in information security from the University of Bradford, U.K., in 2006, and the Ph.D. degree in grid computing from the University of Leeds, U.K., in 2012. In 1998, she joined the Computer Science Department, King Abdulaziz University (KAU), as a Teaching Assistant, where she is currently an Assistant Professor with the Computer Science Department, Faculty of Computing and Information Technology. She is also the Vice Dean of the Faculty of Computing and Information Technology at the female campus of King Abdulaziz University. Her main research interests include information and systems' security, grid/cloud computing, and high-performance computing.

...